

Goal: turn the button inventory into a clean SCSS file plan you can implement.

Overview

For this project, I would keep **all actual button UI in 5-components** and keep 4-objects free of button cosmetics. The only thing that might belong in 4-objects is an optional **layout-only action group object** if you want a reusable wrapper for arranging buttons side by side. That matches your current markup: the real controls are all `c-*` classes, while the wrappers are mostly `o-*` or page/component containers.

Recommended file plan

1) 4-objects/_actions.scss — optional

Use this **only if** you want a reusable layout primitive for action rows.

Put here:

- `.o-actions`
- `.o-actions--center`
- `.o-actions--stack`
- `.o-actions--wrap`

What it should do:

- `display: flex`
- gap / wrapping / alignment only
- no colors
- no borders
- no button padding
- no hover/focus styles

Why: your current hero and section CTA wrappers are structural containers, not buttons themselves. For example, `c-hero__actions` and `c-section-cta` are wrappers around links/buttons, so they could stay component-specific, or be abstracted later into one shared layout object if repetition grows.

If you want to stay simple right now, **skip this file**.

2) 5-components/_button.scss — shared button system

This should be your main reusable button file.

Put here:

- `.c-button`
- `.c-button--primary`
- `.c-button--secondary`
- `.c-button--small`
- `.c-button--large` only if needed later
- `.c-button--with-icon`
- `.c-button__icon-circle`
- `.c-button__icon`
- `.c-button__label`

Used by:

- hero primary CTA
- hero secondary CTA
- homepage “View All Projects” CTA

Those selectors already exist in your markup. The hero uses the full shared button system, including icon variants and size modifiers, while `index.html` uses the simpler primary button.

What belongs here:

- inline-flex alignment
- padding
- border radius
- font weight / size
- background / border / text colors
- hover, focus-visible, active, disabled styles
- icon spacing and alignment

What does **not** belong here:

- page-specific placement
- hero-only spacing around the button group

3) 5-components/_theme-toggle.scss

This should be its own component file, not part of generic buttons.

Put here:

- .c-theme-toggle
- .c-theme-toggle--mobile
- .c-theme-toggle__icon
- .c-theme-toggle__icon--sun
- .c-theme-toggle__icon--moon
- .c-theme-toggle__label

Related attribute hooks you may style carefully here if needed:

- .c-theme-toggle[aria-pressed="true"]
- .c-theme-toggle[aria-pressed="false"]
- .c-theme-toggle[data-theme-toggle]

Why separate it from _button.scss :

- it is a special-purpose control
- it has two icons and a stateful pressed mode
- it appears in both desktop header and mobile menu contexts

This is a standalone button include with its own markup and state behavior.

4) 5-components/_nav-toggle.scss

Make the mobile menu trigger its own file.

Put here:

- .c-nav-toggle
- .c-nav-toggle__icon

Related state/attribute selectors:

- `.c-nav-toggle[aria-expanded="true"]`
- `.c-nav-toggle[aria-expanded="false"]`
- `.c-nav-toggle[data-nav-toggle]`

Why separate:

- it is a header-specific control
- it has toggle state
- it should be easy to find when debugging header/mobile behavior

This button lives in `header.html` and controls `#mobile-menu` through `aria-controls`.

5) 5-components/_mobile-menu.scss

Keep the close button inside the mobile menu component file rather than making a separate close-button file.

Put here:

- `.c-mobile-menu`
- `.c-mobile-menu__inner`
- `.c-mobile-menu__top`
- `.c-mobile-menu__close`
- `.c-mobile-menu__close-symbol`
- `.c-mobile-menu__footer`

Related selectors:

- `.c-mobile-menu[hidden]`
- `.c-mobile-menu[data-mobile-menu]`

Why:

- the close button is tightly tied to the mobile menu shell
- it is not a reusable generic button pattern yet

The close control is clearly part of the mobile menu component.

6) 5-components/_icon-link.scss

Use this for icon-only action links, including social links and CV link styling if they share the same visual language.

Put here:

- `.c-icon-link`
- `.c-icon-link__icon`

Used through `social-links.html` , which is included in both header and footer and inside the mobile menu. Those are not semantic `<button>` elements, but visually they are action controls and belong in components. `footer.html` and `header.html` both reuse `c-social-links` , which delegates to `social-links.html` .

I would then let your social-links component file handle layout/context, while `_icon-link.scss` handles the base clickable UI.

7) 5-components/_social-links.scss

This should remain separate from `_icon-link.scss` .

Put here:

- `.c-social-links`
- `.c-social-links--mobile`
- `.c-social-links--footer`
- `.c-social-links__item`
- `.c-social-links__item--cv`

Why:

- `c-social-links` is a group/list component
- `c-icon-link` is the individual clickable control
- that split is cleaner and more maintainable

This grouping pattern is already implied by the includes in header, footer, and mobile menu.

8) 5-components/_post-share.scss

Keep all share-action styles together here.

Put here:

- .c-post-share
- .c-post-share__list
- .c-post-share__item
- .c-post-share__link
- .c-post-share__icon

JS/state hooks that can stay in this file if needed:

- .js-print-btn
- .js-share-btn

Why:

- this component mixes anchor actions and real buttons
- all of them share the same visual control class: .c-post-share__link

That shared action class is used on both `<a>` and `<button>`, which is a strong sign this is its own self-contained component rather than part of generic buttons.

9) 5-components/_projects-filter.scss

Treat filter chips as their own component, not as generic buttons.

Put here:

- .c-projects-filter
- .c-projects-filter__inner
- .c-projects-filter__chip
- .c-projects-filter__chip.is-active

Why:

- these are navigational filter chips
- they are visually button-like, but semantically links

- they belong to the projects archive UI, not the global button system

Your `projects.html` already uses this as a dedicated component.

10) Leave these out of the shared button system for now

I would **not** move these into `_button.scss` yet:

- `.c-post-card__tag`
- `.c-post__tag`
- `.c-tag`
- `.c-nav__link`

Why:

- they may look pill-like or clickable
- but they belong to their own components semantically and structurally
- abstracting too early usually makes the SCSS harder to reason about

For example, `c-post-card__tag` is part of the post card component, not a reusable site-wide button pattern.

Likewise, `c-tag` in `page.html` is just a content label right now.

Recommended folder result

A clean setup would be:

```
_sass/  
  4-objects/  
    _actions.scss          # optional, layout-only  
  
  5-components/  
    _button.scss  
    _theme-toggle.scss  
    _nav-toggle.scss  
    _mobile-menu.scss  
    _icon-link.scss  
    _social-links.scss  
    _post-share.scss  
    _projects-filter.scss
```

Recommended order to build them

1. _button.scss
2. _theme-toggle.scss
3. _nav-toggle.scss
4. _mobile-menu.scss
5. _icon-link.scss
6. _social-links.scss
7. _post-share.scss
8. _projects-filter.scss

Why this order:

- shared button first
- then header/mobile controls
- then smaller action-link components
- then page-specific chip/filter UI

My concrete recommendation

For your project, I would do this:

- **Do not put actual button styling in 4-objects**
- Put the reusable CTA system in **5-components/_button.scss**
- Keep special controls in their own component files
- Only add a 4-objects/_actions.scss later if you notice repeated layout-only action wrappers

That gives you the cleanest Jekyll-first and ITCSS-correct structure for what you already have in the templates.

Next I can draft the actual **_button.scss selector skeleton** in your ITCSS comment style.